PES UNIVERSITY

Department of CSE (AIML)

8-bit Parallel Prefix Adder

Design and Implementation using Verilog HDL

Mini Project Report

Course: Digital Logic and Computer Design
Course Code: UE25CS251A

Semester: 3
Academic Year: 2026–27
Section: G
Department: CSE(AIML)

Submitted by

Name	SRN
Thanas R	PES1UG25AM427
Prasad M	PES1UG26AM815
Siddharth M R	PES1UG25AM391

1. Problem Statement

To design and implement an **8-bit parallel prefix adder** in Verilog HDL that accepts two 8-bit unsigned operands `A[7:0]` and `B[7:0]` together with a carry-in `Cin`, and produces an 8-bit sum `Sum[7:0]` and a carry-out `Cout`, such that

> *{Cout, Sum} = A + B + Cin*

The design must compute all carries in logarithmic depth ($O(\log_2 n)$ levels) instead of the linear depth of a ripple-carry adder, and must be verified through a simulation test bench.

2. Brief Literature Survey

Fast-adder design has evolved from carry-lookahead to parallel prefix networks, which trade depth, cell count and wiring.

- **Carry-lookahead** (Weinberger & Smith, 1958): carries are computed from generate/propagate signals instead of rippling.
- **Parallel prefix** (Ladner & Fischer, 1980; Sklansky, 1960): carry computation is a prefix problem over an associative operator, giving log-depth trees.
- **Kogge–Stone** (1973): minimum depth ($\log_2 n$), fan-out of 2 and regular wiring, at the cost of many cells and long wires. *Implemented in this project.*
- **Brent–Kung** (1982): fewest cells, but $2 \cdot \log_2 n - 1$ levels.
- **Hybrids** (Han–Carlson, 1987; Knowles, 1999): intermediate depth, cell and wiring trade-offs.

Adder (n = 8)	Carry depth (levels)	Prefix cells	Max. fan-out
Ripple-carry	8 (serial)	–	1
Brent–Kung	5	11	2
Sklansky	3	12	4
Kogge–Stone (this work)	**3**	**17**	**2**

Cell counts are for the prefix network without carry-in.

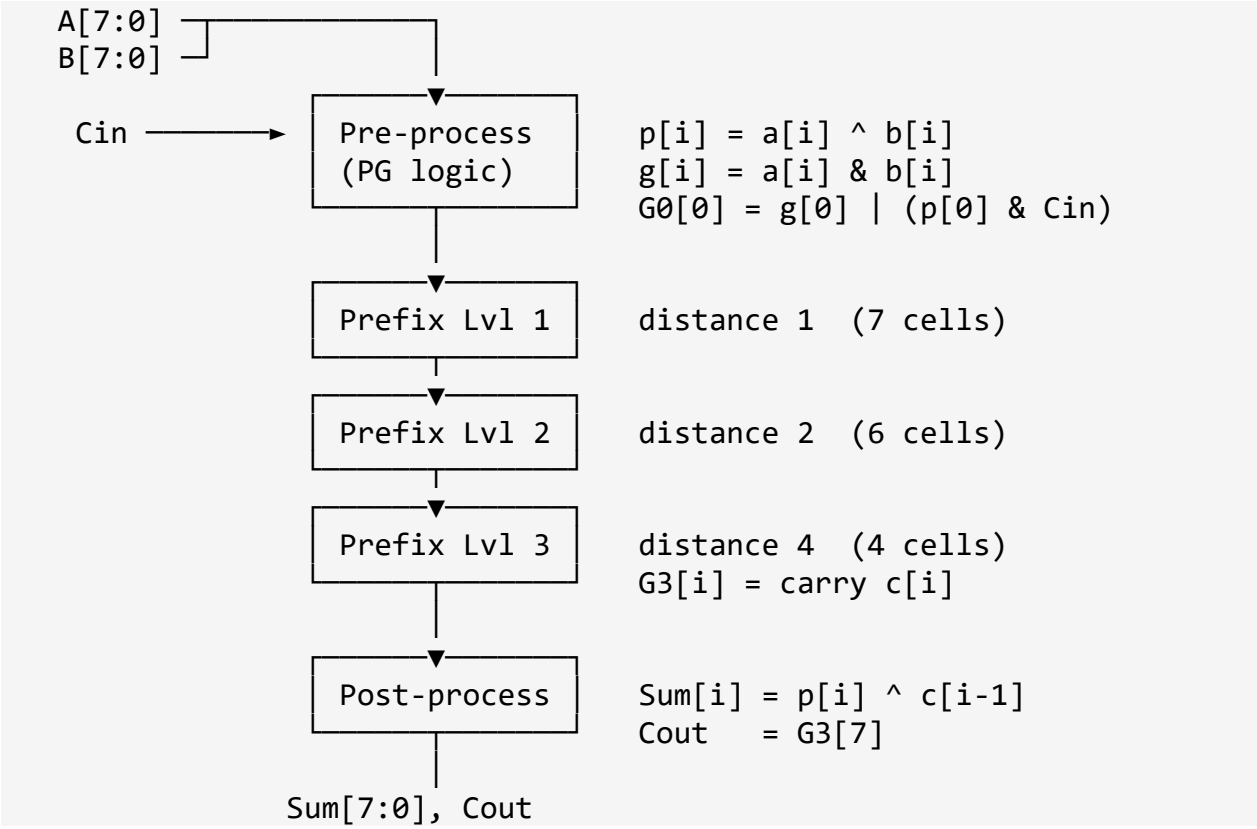
3. Design Methodology

The adder is built in three stages. Because the prefix operator ● is associative, three levels ($\log_2 8$) leave every bit holding G[i:0], which is the carry c_i.

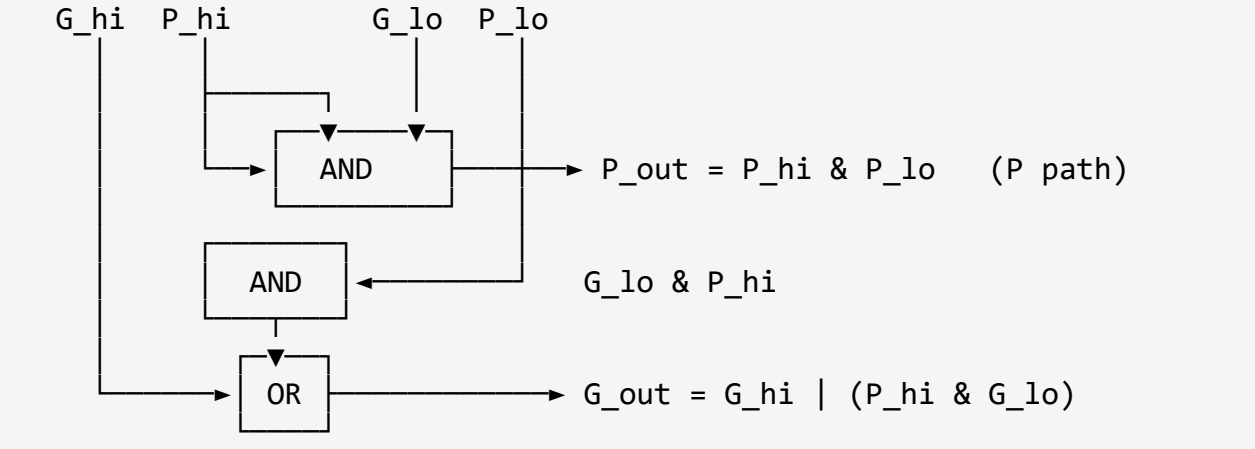
Stage	Operation	Equations
1. Pre-processing	Per-bit propagate and generate; Cin folded into bit 0	$p_i = a_i \oplus b_i$, $g_i = a_i \cdot b_i$, $G_o' = g_o + p_o \cdot Cin$
2. Prefix network	Combine groups at distance 1, 2 and 4	(G, P)hi ● (G, P)lo = (G_hi + P_hi · G_lo, P_hi · P_lo)
3. Post-processing	Sum and carry-out	$Sum_i = p_i \oplus c_{i-1}$ (c_{-1} = Cin), Cout = c_7

4. Circuit Architecture

4.1 Block Diagram



4.2 Prefix Cell (Black Cell)



Each cell is one AND–OR gate pair for G and one AND gate for P, i.e. 3 two-input gates.

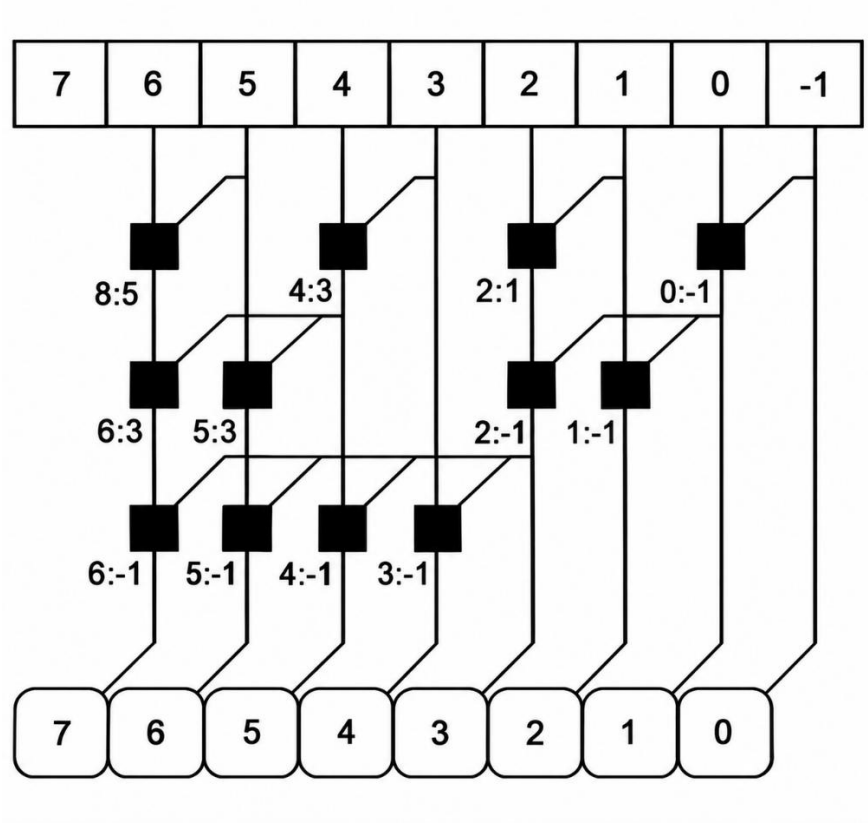
4.3 Prefix Network Span Table

The table shows which bits each node covers after every level. Entries marked ✓ are final (the node already represents [i:0] and no longer needs a cell).

Bit i	After Level 0	After Level 1 (d = 1)	After Level 2 (d = 2)	After Level 3 (d = 4)
0	[0, Cin] ✓	–	–	–
1	[1]	[1:0] ✓	–	–
2	[2]	[2:1]	[2:0] ✓	–
3	[3]	[3:2]	[3:0] ✓	–
4	[4]	[4:3]	[4:1]	[4:0] ✓
5	[5]	[5:4]	[5:2]	[5:0] ✓
6	[6]	[6:5]	[6:3]	[6:0] ✓
7	[7]	[7:6]	[7:4]	[7:0] ✓

Cell count: 7 (level 1) + 6 (level 2) + 4 (level 3) = **17 prefix cells**.

4.4 RTL Schematic



5. Truth Tables

A full truth table of the adder has 2^{17} = 131 072 rows (A, B and Cin), which cannot be listed in a report; it is covered exhaustively by the test bench in Section 10. The truth tables of the building blocks and representative adder rows are given below.

5.1 Generate / Propagate (per bit)

a_i	b_i	$p_i = a_i \oplus b_i$	$g_i = a_i \cdot b_i$	Meaning
0	0	0	0	Kill
0	1	1	0	Propagate
1	0	1	0	Propagate
1	1	0	1	Generate

5.2 Prefix Cell (G output)

G_hi	P_hi	G_lo	G_out = G_hi + P_hi · G_lo
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

The P output is simply P_out = P_hi · P_lo (it is 1 only when both groups propagate).

5.3 Representative Rows of the 8-bit Adder

A (dec)	B (dec)	Cin	Sum (binary)	Cout	A + B + Cin
0	0	0	00000000	0	0
1	1	0	00000010	0	2
15	1	0	00010000	0	16
85	170	0	11111111	0	255
85	170	1	00000000	1	256
255	1	0	00000000	1	256
255	255	1	11111111	1	511

6. Finite State Machine (FSM)

Not applicable. The prefix adder is a purely **combinational** circuit: its outputs depend only on the present inputs and it has no memory elements, states or clock. Hence no state diagram or state table is required.

7. Verilog Code (Design Module)

The design is structural. A reusable `prefix_cell` implements the prefix operator, and `prefix_adder_8` instantiates it through `generate` loops for the three levels.

```
//------------------------------------------------------------------
// Prefix (black) cell : (G,P)hi o (G,P)lo
//------------------------------------------------------------------
module prefix_cell (
    input  g_hi, p_hi,
    input  g_lo, p_lo,
    output g_out, p_out
);
    assign g_out = g_hi | (p_hi & g_lo);
    assign p_out = p_hi & p_lo;
endmodule

//------------------------------------------------------------------
// 8-bit Kogge-Stone parallel prefix adder with carry-in
//------------------------------------------------------------------
module prefix_adder_8 (
    input  [7:0] a,
    input  [7:0] b,
    input        cin,
    output [7:0] sum,
    output       cout
);
    wire [7:0] p, g;          // bit-level propagate / generate
    wire [7:0] G0, P0;        // after pre-processing (cin folded in)
    wire [7:0] G1, P1;        // after level 1 (distance 1)
    wire [7:0] G2, P2;        // after level 2 (distance 2)
    wire [7:0] G3, P3;        // after level 3 (distance 4)

    // Stage 1 : pre-processing
    assign p  = a ^ b;
    assign g  = a & b;
    assign G0 = {g[7:1], g[0] | (p[0] & cin)};
    assign P0 = p;
```

```
    genvar i;

    // Stage 2 : prefix network
    // Level 1 : distance = 1
    generate
        for (i = 0; i < 8; i = i + 1) begin : LVL1
            if (i < 1) begin : PASS
                assign G1[i] = G0[i];
                assign P1[i] = P0[i];
            end else begin : CELL
                prefix_cell u (
                    .g_hi(G0[i]),   .p_hi(P0[i]),
                    .g_lo(G0[i-1]), .p_lo(P0[i-1]),
                    .g_out(G1[i]),  .p_out(P1[i]));
            end
        end
    endgenerate

    // Level 2 : distance = 2
    generate
        for (i = 0; i < 8; i = i + 1) begin : LVL2
            if (i < 2) begin : PASS
                assign G2[i] = G1[i];
                assign P2[i] = P1[i];
            end else begin : CELL
                prefix_cell u (
                    .g_hi(G1[i]),   .p_hi(P1[i]),
                    .g_lo(G1[i-2]), .p_lo(P1[i-2]),
                    .g_out(G2[i]),  .p_out(P2[i]));
            end
        end
    endgenerate

    // Level 3 : distance = 4
    generate
        for (i = 0; i < 8; i = i + 1) begin : LVL3
            if (i < 4) begin : PASS
                assign G3[i] = G2[i];
                assign P3[i] = P2[i];
            end else begin : CELL
                prefix_cell u (
                    .g_hi(G2[i]),   .p_hi(P2[i]),
                    .g_lo(G2[i-4]), .p_lo(P2[i-4]),
                    .g_out(G3[i]),  .p_out(P3[i]));
            end
        end
```

```
    endgenerate

    // Stage 3 : post-processing
    // G3[i] is the carry out of bit i; carry into bit 0 is cin
    assign sum  = p ^ {G3[6:0], cin};
    assign cout = G3[7];
endmodule
```

8. Test Bench

The test bench applies twelve directed vectors (corner cases such as all-ones, long carry chains and Cin = 1) and then an **exhaustive sweep of all 256 × 256 × 2 = 131 072 input combinations**, comparing the DUT with the behavioural expression `A + B + Cin`.

```
`timescale 1ns/1ps

module tb_prefix_adder_8;
    reg  [7:0] a, b;
    reg        cin;
    wire [7:0] sum;
    wire       cout;

    reg  [8:0] expected;
    integer    i, j, k;
    integer    errors, total;

    // Device under test
    prefix_adder_8 dut (.a(a), .b(b), .cin(cin), .sum(sum),
.cout(cout));

    // Apply one vector, check it, optionally print it
    task check;
        input [7:0] ta, tb;
        input       tc;
        input       verbose;
        begin
            a = ta; b = tb; cin = tc;
            #10;
            expected = ta + tb + tc;
            total    = total + 1;
            if ({cout, sum} !== expected) begin
                errors = errors + 1;
                $display("ERROR: %0d + %0d + %0d -> got %0d, expected
%0d",
```

```
                            ta, tb, tc, {cout, sum}, expected);
            end
            if (verbose)
                $display("%3d | %3d |  %b  | %b |  %b   | %3d | %s",
                            ta, tb, tc, sum, cout, {cout, sum},
                            ({cout, sum} === expected) ? "PASS" :
"FAIL");
        end
    endtask

    initial begin
        $dumpfile("prefix_adder_8.vcd");
        $dumpvars(0, tb_prefix_adder_8);
        errors = 0; total = 0;


$display("================================================================
=========");
        $display("  8-bit Parallel Prefix Adder : Directed Tests");

$display("================================================================
=========");
        $display("  A  |  B  | Cin |   Sum    | Cout | A+B+Cin |
Result");
        $display("-------------------------------------------------------
-----------------");

        check(  0,   0, 0, 1);
        check(  1,   1, 0, 1);
        check( 15,   1, 0, 1);
        check( 85, 170, 0, 1);
        check( 85, 170, 1, 1);
        check(255,   1, 0, 1);
        check(255, 255, 1, 1);
        check(128, 128, 0, 1);
        check(100,  27, 0, 1);
        check(200, 100, 1, 1);
        check(127,   1, 0, 1);
        check( 37,  91, 1, 1);

        $display("-------------------------------------------------------
-----------------");
        $display("  Exhaustive test over all input combinations ...");

        for (i = 0; i < 256; i = i + 1)
            for (j = 0; j < 256; j = j + 1)
```

```
                for (k = 0; k < 2; k = k + 1)
                    check(i[7:0], j[7:0], k[0], 0);

        $display("  Vectors applied : %0d", total);
        $display("  Errors found    : %0d", errors);
        if (errors == 0)
            $display("  RESULT: ALL TESTS PASSED");
        else
            $display("  RESULT: TEST FAILED");

$display("============================================================
=========");
        $finish;
    end
endmodule
```

9. Simulation Output and Verification

9.1 Console Output

The table below is the console output expected from the test bench in Section 9 (Icarus Verilog / ModelSim / Vivado). The values were cross-checked against a bit-accurate software model of the same Kogge–Stone network, which matched `A + B + Cin` for all 131 072 input combinations.

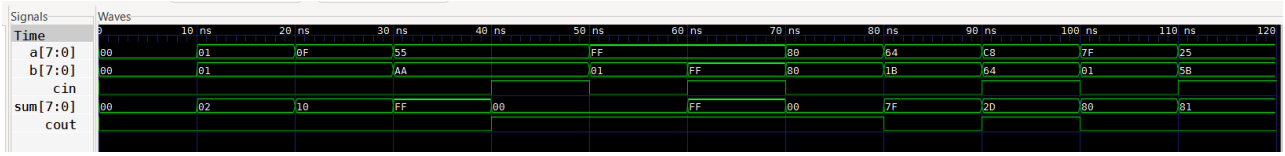
```
======================================================================
  8-bit Parallel Prefix Adder : Directed Tests
======================================================================
  A  |  B  | Cin |   Sum    | Cout | A+B+Cin | Result
----------------------------------------------------------------------
  0 |   0 |  0  | 00000000 |  0   |   0 | PASS
  1 |   1 |  0  | 00000010 |  0   |   2 | PASS
 15 |   1 |  0  | 00010000 |  0   |  16 | PASS
 85 | 170 |  0  | 11111111 |  0   | 255 | PASS
 85 | 170 |  1  | 00000000 |  1   | 256 | PASS
255 |   1 |  0  | 00000000 |  1   | 256 | PASS
255 | 255 |  1  | 11111111 |  1   | 511 | PASS
128 | 128 |  0  | 00000000 |  1   | 256 | PASS
100 |  27 |  0  | 01111111 |  0   | 127 | PASS
200 | 100 |  1  | 00101101 |  1   | 301 | PASS
127 |   1 |  0  | 10000000 |  0   | 128 | PASS
 37 |  91 |  1  | 10000001 |  0   | 129 | PASS
----------------------------------------------------------------------
  Exhaustive test over all input combinations ...
  Vectors applied : 131084
```

```
  Errors found    : 0
  RESULT: ALL TESTS PASSED
========================================================================
```

(131 084 = 12 directed + 131 072 exhaustive vectors.)

9.2 Waveform and Console Screenshots

```
========================================================================
  8-bit Parallel Prefix Adder : Directed Tests
========================================================================
  A  |  B  | Cin |   Sum    | Cout | A+B+Cin | Result
------------------------------------------------------------------------
  0 |   0 |  0  | 00000000 |  0   |   0 | PASS
  1 |   1 |  0  | 00000010 |  0   |   2 | PASS
 15 |   1 |  0  | 00010000 |  0   |  16 | PASS
 85 | 170 |  0  | 11111111 |  0   | 255 | PASS
 85 | 170 |  1  | 00000000 |  1   | 256 | PASS
255 |   1 |  0  | 00000000 |  1   | 256 | PASS
255 | 255 |  1  | 11111111 |  1   | 511 | PASS
128 | 128 |  0  | 00000000 |  1   | 256 | PASS
100 |  27 |  0  | 01111111 |  0   | 127 | PASS
200 | 100 |  1  | 00101101 |  1   | 301 | PASS
127 |   1 |  0  | 10000000 |  0   | 128 | PASS
 37 |  91 |  1  | 10000001 |  0   | 129 | PASS
------------------------------------------------------------------------
  Exhaustive test over all input combinations ...
  Vectors applied : 131084
  Errors found    : 0
  RESULT: ALL TESTS PASSED
========================================================================
```



10. Conclusion

An 8-bit Kogge–Stone parallel prefix adder with carry-in was designed in structural Verilog using a reusable prefix cell and generate loops. The design computes all carries in three prefix levels, and was verified with twelve directed vectors and an exhaustive sweep of all 131 072 combinations with zero errors.